

```

1  ( *
2                                     CS51 Lab 12
3                               Imperative Programming and References
4  *)
5  ( *
6                                     SOLUTION
7  *)
8
9  (*=====
10 Readings:
11
12     This lab builds on material from Chapter 15 of the textbook
13     <http://book.cs51.io>, which should be read before the lab session.
14
15 Objective:
16
17     This lab provides practice with reference types and their use in
18     building mutable data structures and in imperative programming more
19     generally. It also gives further practice in using modules to
20     implement abstract data types.
21
22     There are three total parts to this lab. Please refer to the following
23     files to complete all exercises:
24
25     -> lab12_part1.ml -- Part 1: Fun with references (including `gensym`)
26         lab12_part2.ml -- Part 2: Appending mutable lists
27         lab12_part3.ml -- Part 3: Adding serialization to imperative queues
28     =====*)
29
30  (*=====
31                                     IMPORTANT
32
33     As usual, when implementing functions in this lab, you shouldn't feel
34     beholden to how the definition is introduced in the skeleton code
35     below. For instance, you might want to add a `rec`, or use a different
36     argument list, or no argument list at all but binding to an anonymous
37     function instead. THIS IS ESPECIALLY PERTINENT TO CERTAIN PROBLEMS IN
38     THIS LAB, WHERE THE NORMAL COMPACT NOTATION FOR INTRODUCING FUNCTIONS
39     MAY NOT BE APPROPRIATE.
40     ******)
41
42  (*=====
43 Part 1: Fun with references (including `gensym`)
44
45     .....
46 Exercise 1: Consider a function `incr` that takes an `int ref`
47 argument and has the side effect of incrementing the integer stored in
48 the `ref`. What is an appropriate type for the return value? What
49 should the type for the function as a whole be?
50     .....*)
51
52  (* ANSWER: The function is called for its side effect, not to return

```

```

53     an updated value, so the appropriate return type is `unit`. The type
54     of the function is thus
55
56     int ref -> unit
57 *)
58
59 (*.....
60 Exercise 2: Now implement the `incr` function.
61 .....*)
62
63 let incr (n : int ref) : unit =
64     n := !n + 1 ;;
65
66 (* As it turns out, the `incr` function is provided as part of the
67    `Stdlib` module. Now you know how it can be implemented. *)
68
69 (*.....
70 Exercise 3: Write a function `remember` that returns the last string
71 that it was called with. The first time it is called, it should return
72 the empty string.
73
74     # remember "don't forget" ;;
75     - : string = ""
76     # remember "what was that again?" ;;
77     - : string = "don't forget"
78     # remember "etaoin shrdlu" ;;
79     - : string = "what was that again?"
80
81 (This is probably the least "functional" function ever written!)
82
83 As usual, you shouldn't feel beholden to how the definition is
84 introduced in the skeleton code below.
85 .....*)
86
87 let remember : string -> string =
88     let memory = ref "" in
89     fun (msg : string) ->
90         let previous = !memory in
91         memory := msg;
92         previous ;;
93
94 (* An alternative is to move the `memory` variable outside the
95    function definition, as:
96
97     let memory = ref "" ;;
98
99     let remember msg =
100         let previous = !memory in
101         memory := msg;
102         previous ;;
103
104 But this allows access to the contents of `memory` outside the
105 `remember` function. But this is a failure of the abstraction that
106 `remember` is intended to implement, violating the edict of

```

```

107 prevention. In particular, it would allow someone to modify the
108 `memory` value and so have the `remember` function return a value
109 that was *not* the previous string argument.
110
111 A common error is to start the function as would be typical given
112 its signature:
113
114     let remember msg =
115     ...
116
117 and then introduce the `memory` variable
118
119     let remember msg =
120     let memory = ref "" in
121     ...
122
123 finally finishing with the update and return value
124
125     let remember msg =
126     let memory = ref "" in
127     let previous = !memory in
128     memory := msg;
129     previous ;;
130
131 Unfortunately, this version introduces a new `memory` each time
132 it's called, so always returns the empty string. *)
133
134 (* The `gensym` function (short for "GENerate SYMbol") has a long
135 history dating back to the early days of the programming language
136 LISP. You can find it as early as the 1974 MacLISP manual (page
137 53). <https://url.cs51.io/lisp-gensym>
138
139 (What is LISP you ask? LISP is an untyped functional programming
140 language invented by John McCarthy in 1958, which he based
141 directly on Alonzo Church's lambda calculus. It is one of the most
142 influential programming languages ever devised. You could do worse
143 than spend some time learning the Scheme dialect of LISP, which,
144 by the way, will be made much easier by having learned a typed
145 functional language -- OCaml.)
146
147 The `gensym` function takes a string and generates a new string by
148 suffixing a unique number, which is initially 0 but is incremented
149 each time `gensym` is called.
150
151 For example,
152
153     # gensym "x" ;;
154     - : string = "x0"
155     # gensym "x" ;;
156     - : string = "x1"
157     # gensym "something" ;;
158     - : string = "something2"
159     # gensym (gensym "x") ;;
160     - : string = "x34"

```

```

161
162 There are many uses of `gensym`, one of which is to generate new
163 unique variable names in an interpreter for a programming language. In
164 fact, you'll need `gensym` for just this purpose in completing the
165 final project for CS51, so this is definitely not wasted effort.
166
167 .....
168 Exercise 4: Complete the implementation of `gensym`. As usual, you
169 shouldn't feel beholden to how the definition is introduced in the
170 skeleton code below. (We'll stop mentioning this from now on.)
171 .....*)
172
173 let gensym : string -> string =
174   let suffix = ref 0 in
175   fun str ->
176     let symbol = str ^ string_of_int !suffix in
177     suffix := !suffix + 1;
178     symbol ;;
179
180 (* Taking advantage of the `incr` function implemented above and also
181    available as part of the `Stdlib` module, we can have
182
183    let gensym : string -> string =
184      let suffix = ref 0 in
185      fun str ->
186        let symbol = str ^ string_of_int !suffix in
187        incr suffix;
188        symbol ;;
189
190    An alternative implementation performs the incrementation before
191    the string formation, so that the return value need not be stored
192    explicitly. But the initial value for the counter must start one
193    earlier.
194
195    let gensym : string -> string =
196      let suffix = ref ~-1 in
197      fun str ->
198        incr suffix;
199        str ^ string_of_int !suffix ;;
200    *)

```